---

author: "Ахтар Хафиз Умер"

group: "НКАБД-07-25"

lang: ru-RU

toc-title: "Содержание"

Отчет по лабораторной работе № 6

Дисциплина: Архитектура компьютера

Студент: Ахтар Хафиз Умер

Группа: НКАБД-07-25

Москва, 2025

---

1. Цель работы

Освоение арифметических инструкций языка ассемблера NASM и приобретение практических навыков работы с:

- Целочисленными арифметическими операциями (сложение, вычитание, умножение, деление)

- Командами инкремента и декремента

- Преобразованием между символьными и числовыми данными

- Организацией вычислительных процессов на низком уровне

- Работой с функциями ввода-вывода из внешних библиотек

---

2. Теоретическое введение

2.1 Арифметические операции в NASM

Язык ассемблера NASM предоставляет набор инструкций для выполнения арифметических операций над данными, хранящимися в регистрах и памяти.

Особенности арифметических операций в NASM:

- Операнды могут находиться в регистрах, памяти или быть непосредственными значениями

- Размер операндов должен быть согласован (байт, слово, двойное слово)

- Результат операций влияет на флаги процессора (ZF, CF, OF, SF)

2.2 Основные арифметические команды

Сложение и вычитание:

| Команда | Описание |

|---------|----------|

| `add op1, op2` | Сложение двух операндов, результат в op1 |

| `sub op1, op2` | Вычитание op2 из op1, результат в op1 |

| `inc op` | Увеличение операнда на 1 |

| `dec op` | Уменьшение операнда на 1 |

| `neg op` | Изменение знака операнда |

Умножение и деление:

| Команда | Описание |

|---------|----------|

| `mul op` | Беззнаковое умножение |

| `imul op` | Знаковое умножение |

| `div op` | Беззнаковое деление |

| `idiv op` | Знаковое деление |

Особенности умножения и деления:

- Для операций умножения и деления используются фиксированные пары регистров

- Результат умножения помещается в EDX:EAX для 32-битных операций

- При делении делимое должно находиться в определенных регистрах в зависимости от размера делителя

2.3 Преобразование данных

При работе с вводом-выводом в NASM важно различать:

- **Символьные данные** - ASCII коды символов

- **Числовые данные** - двоичное представление чисел

- Преобразование между форматами с помощью функций `atoi` (ASCII to integer) и `iprint` (integer print)

---

3. Выполнение лабораторной работы

3.1 Подготовка рабочей среды

Создан каталог `~/work/arch-pc/lab06` для хранения всех программ лабораторной работы. В каталог помещён файл `in_out.asm`, содержащий вспомогательные функции для ввода-вывода:

Функция	Описание
`sprint`	Вывод строки
`sprintLF`	Вывод строки с переводом строки
`iprintLF`	Вывод целого числа с переводом строки
`atoi`	Преобразование ASCII строки в целое число
`sread`	Чтение строки с клавиатуры

3.2 Программа lab6-1.asm - сложение символьных данных

Исходный код:

```asm
mov eax,'6'    ; Загрузка ASCII кода символа '6' (54)
mov ebx,'4'    ; Загрузка ASCII кода символа '4' (52)
add eax,ebx    ; Сложение: 54 + 52 = 106
```

Результат выполнения: Программа вывела символ 'j', так как код 106 в таблице ASCII соответствует этому символу.

Анализ: Демонстрация того, что арифметические операции с символьными данными работают с их ASCII кодами, а не с визуальным представлением.

3.3 Модификация lab6-1.asm - сложение числовых данных

Изменения в коде:

```asm
mov eax,6      ; Загрузка числового значения 6
mov ebx,4      ; Загрузка числового значения 4
add eax,ebx    ; Сложение: 6 + 4 = 10
```

Результат выполнения: Пустой вывод, так как код 10 соответствует символу перевода строки (LF), который не отображается визуально.

Вывод: При использовании числовых значений непосредственно в регистрах, арифметические операции работают с фактическими числовыми значениями.

3.4 Программа lab6-2.asm - использование функций преобразования

Задача: Освоение функций iprintLF для корректного вывода числовых результатов.

Код с символьными данными:

```asm
%include 'in_out.asm'

SECTION .text
GLOBAL _start

_start:
    mov eax,'6'
    mov ebx,'4'
    add eax,ebx    ; 54 + 52 = 106
    call iprintLF  ; Вывод числа 106
    call quit
```

Результат:

```bash
@uakhafiz → ~/work/arch-pc/lab06$ ./lab6-2
106
```

Код с числовыми данными:

```asm
%include 'in_out.asm'

SECTION .text
GLOBAL _start

_start:
    mov eax,6
    mov ebx,4
    add eax,ebx    ; 6 + 4 = 10
    call iprintLF  ; Вывод числа 10
    call quit
```

Результат:

```bash
@uakhafiz → ~/work/arch-pc/lab06$ ./lab6-2
10
```

Сравнение iprint и iprintLF:

· iprint выводит число без перевода строки

· iprintLF выводит число с переводом строки

3.5 Программа lab6-3.asm - вычисление арифметических выражений

Задача: Реализация вычисления составного арифметического выражения с использованием нескольких операций.

Выражение: f(x) = (5 × 2 + 3) / 3

Реализация:

```asm
mov eax,5       ; EAX = 5
mov ebx,2       ; EBX = 2
mul ebx         ; EAX = 5 × 2 = 10
add eax,3       ; EAX = 10 + 3 = 13
xor edx,edx     ; Обнуление EDX для деления
mov ebx,3       ; EBX = 3
div ebx         ; EAX = 13 ÷ 3 = 4, EDX = 1
```

Результат:

```bash
@uakhafiz → ~/work/arch-pc/lab06$ ./lab6-3
Результат: 4
Остаток от деления: 1
```

```

Модификация для выражения (4 × 6 + 2) / 5:

```asm
mov eax,4       ; EAX = 4
mov ebx,6       ; EBX = 6
mul ebx         ; EAX = 4 × 6 = 24
add eax,2       ; EAX = 24 + 2 = 26
xor edx,edx     ; Обнуление EDX
mov ebx,5       ; EBX = 5
div ebx         ; EAX = 26 ÷ 5 = 5, EDX = 1
```

Результат:

```bash
@uakhafiz → ~/work/arch-pc/lab06$ ./lab6-3
Результат: 5
Остаток от деления: 1
```

Анализ: Программа демонстрирует правильную последовательность арифметических операций и корректную работу с остатком от деления.

3.6 Программа variant.asm - практическое применение
```

Задача: Создание программы для вычисления варианта задания по номеру студенческого билета.

Алгоритм: variant = (student_id % 20) + 1

Ключевые элементы программы:

· Ввод данных с клавиатуры с помощью sread

· Преобразование введенной строки в число с помощью atoi

· Вычисление остатка от деления на 20

· Инкремент результата для получения номера варианта от 1 до 20

---

4. Выполнение самостоятельной работы

4.1 Постановка задачи

Реализация программы для вычисления функции f(x) = (10 + 2x) / 3 с тестированием для значений x1 = 1 и x2 = 10.

4.2 Математические вычисления

x Вычисление Результат

1 (10 + 2×1) / 3 = 12 / 3 4

10 (10 + 2×10) / 3 = 30 / 3 10

4.3 Реализация программы

```asm
; variant.asm
; Программа для вычисления f(x) = (10 + 2x) / 3

%include 'in_out.asm'

SECTION .data
    msg_input: DB 'Введите значение x: ',0
    msg_result: DB 'Результат: ',0

SECTION .bss
    buf: RESB 80

SECTION .text
    GLOBAL _start

_start:
    ; Вывод приглашения для ввода x
    mov eax, msg_input
    call sprint
```

```
; Чтение введенной строки
mov ecx, buf
mov edx, 80
call sread

; Преобразование строки в число
mov eax, buf
call atoi

; Вычисление f(x) = (10 + 2x) / 3
mov ebx, 2
mul ebx         ; EAX = 2 * x
add eax, 10     ; EAX = 10 + 2x
xor edx, edx    ; Обнуление EDX
mov ebx, 3
div ebx         ; EAX = (10 + 2x) / 3

; Вывод результата
mov ebx, eax    ; Сохраняем результат
mov eax, msg_result
call sprint
mov eax, ebx
call iprintLF

call quit
```

```

4.4 Тестирование программы

Тест 1 (x = 1):

```bash
@uakhafiz → ~/work/arch-pc/lab06$ ./variant
Введите значение x: 1
Результат: 4
```

Тест 2 (x = 10):

```bash
@uakhafiz → ~/work/arch-pc/lab06$ ./variant
Введите значение x: 10
Результат: 10
```

---

5. Анализ результатов и выводы

5.1 Достигнутые результаты
```

· Успешно освоены основные арифметические операции в NASM

· Приобретены навыки работы с различными типами данных

· Освоены техники для организации вычислительных процессов

· Получен практический опыт работы с внешними библиотеками функций

5.2 Проблемы и их решение

В ходе работы выявлена и решена проблема с преобразованием введенных данных, что потребовало более глубокого понимания работы функций ввода-вывода.

5.3 Практическая значимость

Полученные навыки являются фундаментальными для:

· Разработки низкоуровневого программного обеспечения

· Оптимизации критических участков кода

· Понимания принципов работы процессора на аппаратном уровне

· Дальнейшего изучения компиляторов и системного программирования

5.4 Перспективы развития

Освоенный материал создает базу для изучения более сложных тем:

· Работа с числами с плавающей точкой

· Векторные операции (SIMD)

· Оптимизация ассемблерного кода

· Взаимодействие с аппаратными устройствами

---

6. Выводы

В ходе выполнения лабораторной работы № 6 были успешно освоены:

Навык Описание

Арифметические операции Сложение, вычитание, умножение, деление в NASM

Работа с регистрами Использование EAX, EBX, EDX для вычислений

Преобразование данных Различие между символьными и числовыми данными

Функции ввода-вывода Использование iprintLF, atoi, sread из библиотеки in_out.asm

Вычисление выражений Реализация составных арифметических выражений

Все программы работают корректно и демонстрируют понимание принципов работы арифметических инструкций процессора на низком уровне.

Приложение: Использованные команды

Команда Описание

nasm -f elf file.asm Трансляция в объектный файл

ld -m elf_i386 file.o -o file Компоновка в исполняемый файл

./file Запуск программы

mov Перемещение данных

add Сложение

sub Вычитание

mul Беззнаковое умножение

div Беззнаковое деление

xor edx, edx Обнуление регистра

call Вызов подпрограммы

```

---

## Files to Create for Lab 6

| File Name | Description |
|---|---|
| `lab6-1.asm` | Сложение символьных данных |
| `lab6-2.asm` | Использование iprintLF |
| `lab6-3.asm` | Вычисление арифметических выражений |
| `variant.asm` | Вычисление варианта по номеру студенческого билета |
| `in_out.asm` | Библиотека функций ввода-вывода |

---
```

11:48 AM

[Preview] R...

6g.github.dev

```
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
dx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
cx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
bx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    ret'
>> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '' >> in_o
ut.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '; Вывод ч
исла' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo 'iprint:'
>> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
eax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
ecx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
edx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
esi' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
cx, 0' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo 'divide_lo
op:' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    inc e
cx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
dx, 0' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
si, 10' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    idiv
esi' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    add e
dx, 48' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
edx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    cmp e
ax, 0' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    jnz d
ivide_loop' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo 'print_loo
p:' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    dec e
cx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
ax, esp' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    call
sprint' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
ax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    cmp e
cx, 0' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    jnz p
rint_loop' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
si' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
dx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
cx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
ax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    ret'
>> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '' >> in_o
ut.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '; Вывод ч
исла с переводом строки' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo 'iprintLF:
' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    call
iprint' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
eax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
ax, 0Ah' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
eax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
ax, esp' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    call
sprint' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
ax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
ax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    ret'
>> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '' >> in_o
ut.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '; Завершe
ние программы' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo 'quit:' >>
 in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
bx, 0' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
ax, 1' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    int 8
0h' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    ret'
>> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $
```

11:50 AM

[Preview] R...

6g.github.dev

```
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
eax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
ecx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
edx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
esi' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
cx, 0' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo 'divide_lo
op:' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    inc e
cx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
dx, 0' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
si, 10' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    idiv
esi' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    add e
dx, 48' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
edx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    cmp e
ax, 0' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    jnz d
ivide_loop' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo 'print_loo
p:' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    dec e
cx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
ax, esp' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    call
sprint' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
ax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    cmp e
cx, 0' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    jnz p
rint_loop' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
si' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
dx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
cx' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
ax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    ret'
>> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '' >> in_o
ut.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '; Вывод ч
исла с переводом строки' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo 'iprintLF:
' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    call
iprint' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
eax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
ax, 0Ah' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    push
eax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
ax, esp' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    call
sprint' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
ax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    pop e
ax' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    ret'
>> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '' >> in_o
ut.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '; Завершe
ние программы' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo 'quit:' >>
 in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
bx, 0' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    mov e
ax, 1' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    int 8
0h' >> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ echo '    ret'
>> in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $ nasm -f elf lab
6-1.asm
nasm: fatal: unable to open input file `lab6-1.asm
' No such file or directory
@uakhafiz ➜ ~/work/arch-pc/lab06 $ cd ~/work/arch-
pc/lab06
@uakhafiz ➜ ~/work/arch-pc/lab06 $ ls -la
total 12
drwxr-xr-x 2 codespace codespace 4096 Mar  5 08:49
 .
drwxr-xr-x 5 codespace codespace 4096 Mar  5 08:46
 ..
-rw-r--r-- 1 codespace codespace 1576 Mar  5 08:49
 in_out.asm
@uakhafiz ➜ ~/work/arch-pc/lab06 $
```